Support élève (Thème 3)

Objectifs : Thème 3 : Structures linéaires

Dans ce chapitre, nous allons étudier différentes structures de données linéaires.

Nous chercherons notamment à comprendre :

- ce qu'est une structure de données abstraite ;
- comment fonctionne une liste ;
- comment implémenter une liste chaînée ;
- comment fonctionne une file et le principe FIFO ;
- comment fonctionne une pile et le principe LIFO ;
- comment implémenter ces structures à l'aide de classes Python ;
- comment utiliser une pile pour implémenter une file et inversement.

À la fin de ce chapitre, vous devrez être capables de choisir une structure de données adaptée à un problème, de simuler les opérations qui lui sont appliquées et d'implémenter les opérations fondamentales d'une liste chaînée, d'une pile et d'une file en Python.

Exercice 1 : Ba-liste-ique

1. Voici une série d'instructions. Les instructions ci-dessous s'enchaînent. Expliquez ce qui se passe à chacune des étapes :

```
L = vide()
estVide(L)
ajouteEnTete(3,L)
estVide(L)
ajouteEnTete(5,L)
ajouteEnTete(8,L)
t = supprEnTete(L)
L1 = vide()
L2 = cons(8, cons(5, cons(3, L1)))
```

2. Voici une série d'instructions. Les instructions ci-dessous s'enchaînent. Expliquez ce qui se passe à chacune des étapes :

```
L = vide()
ajoutEnTete(10,L)
ajoutEnTete(9,L)
ajoutEnTete(7,L)
L1 = vide()
L2 = cons(5, cons(4, cons(3, cons(2, cons(1, cons(0,L1))))))
```

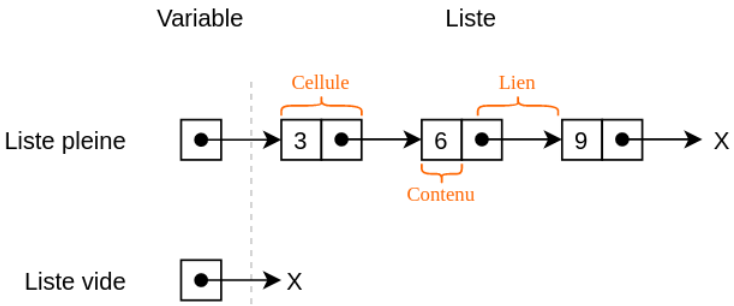
Activité 1 : Implémentation

1. Écrire une classe `Cellule`, possédant deux attributs : `contenu` (la valeur de la cellule) et `suivante` (une instance de la classe `Cellule`).

On pourra alors créer une liste chaînée de la façon suivante :

```
lst = Cellule(3, Cellule(5, Cellule(1, None)))
```

La liste construite sera de la forme suivante :



2. Écrire les lignes de code permettant d'afficher la valeur de toutes les cellules.

Exercice 2 : Ecris liste oire

On dispose de l'interface offrant les fonctionnalités suivantes :

1. `ListeC()` : crée une liste vide ;
2. `est_vide()` : indique si la liste est vide ;
3. `ajoute_tete()` : insère un élément en tête de liste ;
4. `renvoie_tete()` : renvoie la valeur de l'élément en tête de liste et le supprime de la liste.

1. On considère l'enchaînement d'opérations ci-dessous. Écrire à chaque étape l'état de la liste `lst` et la valeur éventuellement renvoyée.

```
lst = ListeC()
lst.ajoute_tete(3)
lst.ajoute_tete(5)
lst.ajoute_tete(1)
lst.renvoie_tete()
lst.est_vide()
lst.ajoute_tete(2)
lst.renvoie_tete()
lst.renvoie_tete()
lst.renvoie_tete()
lst.est_vide()
```

Activité 2 : Classe à Dallas

1. Ajouter une méthode `__str__` à votre classe `Cellule`. Elle affichera le contenu de la cellule et celui des suivantes si elles existent.

Elle renverra donc : 3 -> 5 -> 1 pour la liste `lst` créée précédemment.

2. Compléter la classe `ListeC` suivante :

```
class ListeC:
    def __init__(self):
        self.tete = None  # Liste vide

    def ajoute_tete(self, valeur):
        '''
        Créer une nouvelle cellule contenant la valeur et l'ajouter en tête.
        '''
        ...
        ...
        ...

    def est_vide(self):
        '''
        Indiquer si la liste est vide ou non.
        @return : (bool) True si vide, False sinon
        '''
        ...
        ...
        ...

    def renvoie_tete(self):
        '''
        Renvoyer le contenu de la tête de la liste et la retirer
        '''
```

```
                ...
                ...
                ...
                ...
                ...
                ...
                ...
                ...
                ...

            def __str__(self):  # affichage de la liste
                return str(self.tete) if not self.est_vide() else ""
```

3. Écrire la suite d'instructions permettant de vérifier votre réponse à l'exercice 2.

Exercice 3 : File dans ta chambre

Dans les différentes questions, vous considérez pouvoir utiliser les opérations de la SDA donnée ci-dessous.

```
class File:
    def __init__(self):
        ...

    def enfiler(self, x):
        ...

    def defiler(self):
        ...

    def est_vide(self):
        ...
```

1. Écrire une suite d'instructions permettant de créer une instance de la classe `File` affectée à la variable `ma_file` contenant les éléments `1`, `2` et `3` insérés dans cet ordre.

2. Donner l'affichage obtenu après les appels suivants.

```
# On considère ma_deuxieme_file bien initialisée comme une file vide
ma_deuxieme_file.enfiler(5)
ma_deuxieme_file.enfiler(4)
ma_deuxieme_file.enfiler(3)
ma_deuxieme_file.defiler()
ma_deuxieme_file.defiler()
ma_deuxieme_file.enfiler(4)
ma_deuxieme_file.enfiler(ma_deuxieme_file.defiler())
```

3. Donner une implémentation de l'opération `taille` renvoyant le nombre d'éléments d'une file en utilisant seulement les éléments Python de base et les opérations de la SDA données précédemment.

4. Donner une implémentation de l'opération `inverser` permettant d'inverser les éléments de la file.
La file d'ordre `1`, `2` et `3` deviendra donc la file d'ordre `3`, `2` et `1` après exécution de l'opération `inverser`.

Activité 3 : File un mauvais coton

1. Implémenter une structure de données abstraite (SDA) de type file en Python, en utilisant une liste (list) pour stocker les éléments.

Compléter les opérations suivantes :

- `__init__` : initialiser une file vide ;
- `enfiler(x)` : ajouter l'élément x à la fin de la file ;
- `defiler()` : retirer et renvoyer l'élément situé en tête de la file ;
- `est_vide()` : indiquer si la file est vide.

2. Écrire un bref jeu de tests permettant de vérifier le bon fonctionnement de votre implémentation. Vous testerez notamment l'ajout d'éléments, leur retrait et le respect du principe FIFO (First In, First Out).

Exercice 4 : Pile à l'heure

Dans les différentes questions, vous considérez pouvoir utiliser les opérations de la SDA donnée ci-dessous.

```
class Pile:
    def __init__(self):
        ...

```

```
    def empiler(self, x):
        ...

    def depiler(self):
        ...

    def est_vide(self):
        ...
```

1. Écrire une suite d'instructions permettant de créer une instance de la classe `Pile`, affectée à la variable `ma_pile`, contenant les éléments 3, 2 et 1 insérés dans cet ordre.

2. Donner l'affichage obtenu après les appels suivants.

```
# On considère ma_deuxieme_pile bien initialisée comme une pile vide
ma_deuxieme_pile.empiler(20)
ma_deuxieme_pile.empiler(7)
ma_deuxieme_pile.empiler(68)
ma_deuxieme_pile.depiler()
ma_deuxieme_pile.depiler()
ma_deuxieme_pile.empiler(45)
ma_deuxieme_pile.empiler(ma_deuxieme_pile.depiler())
```

3. Donner une implémentation de l'opération `taille` renvoyant le nombre d'éléments d'une pile en utilisant seulement les éléments Python de base et les opérations de la SDA données précédemment.

4. Donner une implémentation de l'opération `inverser` permettant d'inverser les éléments de la pile.

La pile contenant les éléments 1, 2 et 3 (avec 1 comme sommet) deviendra donc la pile contenant les éléments 3, 2 et 1 (avec 3 comme sommet) après exécution de l'opération `inverser`.

Activité 4 : Pile Poil

1. Implémenter une structure de données abstraite (SDA) de type pile en Python, en utilisant une liste (list) pour stocker les éléments.

Compléter les opérations suivantes :

- `__init__` : initialiser une pile vide ;
- `empiler(x)` : ajouter l'élément x au sommet de la pile ;
- `depiler()` : retirer et renvoyer l'élément situé au sommet de la pile ;
- `est_vide()` : indiquer si la pile est vide.

2. Écrire un bref jeu de tests permettant de vérifier le bon fonctionnement de votre implémentation. Vous testerez notamment l'ajout d'éléments, leur retrait et le respect du principe LIFO (Last In, First Out).

Activité 5 : Dupile et Dufile

Vous possédez une SDA de la file semblable à celle présentée dans l'exercice 3.

1. Grâce à la SDA de la file, implémenter une classe `Pile` en utilisant les files.

Vous possédez une SDA de la pile semblable à celle présentée dans l'exercice 4.

2. Grâce à la SDA de la pile, implémenter une classe `File` en utilisant les piles.



Exercice 5 : Midi Pile

On crée une classe Pile qui modélise la structure d'une pile d'entiers. Le constructeur de la classe initialise une pile vide. La définition de cette classe sans l'implémentation de ses méthodes est donnée ci-dessous.

```
class Pile:
    def __init__(self):
        """Initialise la pile comme une pile vide."""

    def est_vide(self):
        """Renvoie True si la liste est vide, False sinon."""

    def empiler(self, e):
        """Ajoute l'élément e sur le sommet de la pile, ne renvoie rien."""

    def depiler(self):
        """Retire l'élément au sommet de la pile et le renvoie."""

    def nb_elements(self):
        """Renvoie le nombre d'éléments de la pile."""

    def afficher():
        """Affiche de gauche à droite les éléments de la pile, du fond
        de la pile vers son sommet. Le sommet est alors l'élément
        affiché le plus à droite. Les éléments sont séparés par une
        virgule. Si la pile est vide la méthode affiche "pile vide"."""
```

1.

a. Écrire une suite d'instructions permettant de créer une instance de la classe `Pile`, affectée à une variable `pile1`, contenant les éléments 7, 5 et 2 insérés dans cet ordre.
Ainsi, à l'issue de ces instructions, l'instruction `pile1.afficher()` produit l'affichage : 7, 5, 2

b. Donner l'affichage produit après l'exécution des instructions suivantes.

```
element1 = pile1.depiler()
pile1.empiler(5)
pile1.empiler(element1)
pile1.afficher()
```

2.

On donne la fonction `mystere` suivante.

```
def mystere(pile, element) :
    pile2 = Pile()
    nb_elements = pile.nb_elements()
    for i in range(nb_elements) :
        elem = pile.depiler()
        pile2.empiler(elem)
        if elem == element :
            return pile2
    return pile2
```

a. Dans chacun des quatre cas suivants, quel est l'affichage obtenu dans la console ?

1.
```
>>>pile.afficher()
7, 5, 2, 3
>>>mystere(pile, 2).afficher()
```

2.
```
>>>pile.afficher()
7, 5, 2, 3
>>>mystere(pile, 9).afficher()
```

3.
```
>>>pile.afficher()
7, 5, 2, 3
>>>mystere(pile, 3).afficher()
```

4.
```
>>>pile.est_vide()
True
>>>mystere(pile, 3).afficher()
```

b. Expliquer ce que permet d'obtenir la fonction `mystere`.

3. Écrire une fonction `etendre(pile1, pile2)` qui prend en arguments deux objets `Pile`, appelés `pile1` et `pile2`, et qui modifie `pile1` en lui ajoutant les éléments de `pile2` rangés dans l'ordre inverse.

Cette fonction ne renvoie rien.

On donne ci-dessous les résultats attendus pour certaines instructions.

```
>>>pile1.afficher()
7, 5, 2, 3
>>>pile2.afficher()
1, 3, 4
>>>etendre(pile1, pile2)
>>>pile1.afficher()
7, 5, 2, 3, 4, 3, 1
>>>pile2.est_vide()
True
```

4. Écrire une fonction `supprime_toutes_occurences(pile, element)` qui prend en arguments un objet `Pile`, appelé `pile`, et un élément `element`, et qui supprime toutes les occurrences de `element` dans `pile`.

On donne ci-dessous les résultats attendus pour certaines instructions.

```
>>>pile.afficher()
7, 5, 2, 3, 5
>>>supprime_toutes_occurences(pile, 5)
>>>pile.afficher()
7, 2, 3
```



Exercice 6 : File du rasoir

Un supermarché met en place un système de passage automatique en caisse.

Un client scanne les articles à l'aide d'un scanner de codes-barres au fur et à mesure qu'il les ajoute dans son panier.

Les articles s'enregistrent alors dans une structure de données.

La structure de données utilisée est une file définie par la classe `Panier`, avec les primitives habituelles sur la structure de file.

Pour faciliter la lecture, le code de la classe `Panier` n'est pas écrit.

```
class Panier:
    def __init__(self):
        """Initialise la file comme une file vide."""

    def est_vide(self):
        """Renvoie True si la file est vide, False sinon."""

    def enfiler(self, e):
        """Ajoute l'élément e en dernière position de la file, ne renvoie rien."""

    def defiler(self):
        """Retire le premier élément de la file et le renvoie."""
```

Le panier d'un client sera représenté par une file contenant les articles scannés.

Les articles sont représentés par des tuples `(code_barre, designation, prix, horaire_scan)` où :

- `code_barre` est un nombre entier identifiant l'article ;
- `designation` est une chaîne de caractères qui pourra être affichée sur le ticket de caisse ;
- `prix` est un nombre décimal donnant le prix d'une unité de cet article ;
- `horaire_scan` est un nombre entier de secondes permettant de connaître l'heure où l'article a été scanné.

1. On souhaite ajouter un article dont le tuple est le suivant :

```
(31002, "café noir", 1.50, 50525)
```

Écrire le code utilisant une des quatre méthodes ci-dessus permettant d'ajouter l'article à l'objet de classe `Panier`, appelé `panier1`.

2. On souhaite définir une méthode `remplir(panier_temp)` dans la classe `Panier` permettant de remplir la file avec tout le contenu d'un autre `panier_temp`, qui est un objet de type `Panier`.

Recopier et compléter le code de la méthode `remplir` en remplaçant chaque `...` par la primitive de file qui convient.

```
def remplir(self, panier_temp):
while not panier_temp. ......... :
    article = panier_temp. ...........
    self. ...........(article)
```

3. Pour que le client puisse connaître à tout moment le montant de son panier, on souhaite ajouter une méthode `prix_total()` à la classe `Panier`, qui renvoie la somme des prix de tous les articles présents dans le panier.

Écrire le code de la méthode `prix_total()`.

Attention : après l'appel de cette méthode, le panier devra toujours contenir ses articles.

4. Le magasin souhaite connaître, pour chaque client, la durée de ses achats.

Cette durée sera obtenue en faisant la différence entre le champ `horaire_scan` du dernier article scanné et le champ `horaire_scan` du premier article scanné dans le panier du client.

Un panier vide renverra une durée égale à zéro.

On pourra accepter que le panier soit vide après l'appel de cette méthode.

Écrire une méthode `duree_courses()` de la classe `Panier` qui renvoie cette durée.

Exercice 7 : File du rasoir

Les interfaces des structures de données abstraites `Pile` et `File` sont proposées ci-dessous.

On utilisera uniquement les méthodes présentées ci-dessous :

Structure de données abstraite : Pile
Utilise : Élément, Booléen
Opérations : • creer_pile_vide : ∅ → Pile creer_pile_vide() renvoie une pile vide • est_vide : Pile → Booléen est_vide(pile) renvoie True si pile est vide, False sinon • empiler : Pile, Élément → ∅ empiler(pile, element) ajoute element à la pile pile • depiler : Pile → Élément depiler(pile) renvoie l'élément au sommet de la pile pile en le retirant de la pile pile

Structure de données abstraite : File
Utilise : Élément, Booléen
Opérations : • creer_file_vide : ∅ → File creer_file_vide() renvoie une file vide • est_vide : File → Booléen est_vide(file) renvoie True si file est vide, False sinon • enfiler : File, Élément → ∅ enfiler(file, element) ajoute element dans la file file • defiler : File → Élément defiler(file) renvoie l'élément au sommet de la file file en le retirant de la file file

1. a. On considère la file F suivante :

enfilement → "rouge" "vert" "jaune" "rouge" "jaune" →défilement

Quel sera le contenu de la pile P et de la file F après l'exécution du programme Python suivant ?

```
1       P = creer_pile_vide()
2       while not (estvide(F)):
3           empiler(P, defiler(F))
```

b. Créer une fonction `taille_file` qui prend en paramètre une file `F` et qui renvoie le nombre d'éléments qu'elle contient.

Après l'appel de cette fonction, la file `F` doit avoir retrouvé son état d'origine.

2. Écrire une fonction `former_pile` qui prend en paramètre une file `F` et qui renvoie une pile `P` contenant les mêmes éléments que la file.

Le premier élément sorti de la file devra se trouver au sommet de la pile ; le deuxième élément sorti de la file devra se trouver juste en dessous du sommet, etc.

Exemple : si F = ”rouge” ”vert” ”jaune” ”rouge” ”jaune” alors l'appel `former_pile(F)` va renvoyer la pile P ci-dessous :

$$P = \left| \begin{array}{l} \text{"jaune"} \\ \text{"rouge"} \\ \text{"jaune"} \\ \text{"vert"} \\ \text{"rouge"} \end{array} \right|$$

3. Écrire une fonction `nb_elements` qui prend en paramètres une file `F` et un élément `elt`, et qui renvoie le nombre de fois où `elt` est présent dans la file `F`.

Après l'appel de cette fonction, la file `F` doit avoir retrouvé son état d'origine.

4. Écrire une fonction `verifier_contenu` qui prend en paramètres une file `F` et trois entiers : `nb_rouge`, `nb_vert` et `nb_jaune`.

Cette fonction renvoie le booléen `True` si :

- `"rouge"` apparaît au plus `nb_rouge` fois dans la file `F` ;
- `"vert"` apparaît au plus `nb_vert` fois dans la file `F` ;
- `"jaune"` apparaît au plus `nb_jaune` fois dans la file `F`.

Elle renvoie `False` sinon. On pourra utiliser les fonctions précédentes.